

Claude Code for Real Estate Operators

*Everything you need to start using AI
to run your business by next week*



`github.com/dustins89/cg-marketing-cli`

WHAT'S IN THIS PACKET

A 15-minute install · The "day one" guardrail prompt

A starter prompt to build your own marketing CLI

What to learn before letting Claude touch a Salesforce org

*Presented at Collective Genius Premier · **Dustin Singer***

IG @dustinbuyshouses · `dustin@dustinbuyshouses.net`

Step 1 — The 15-minute install

Open your Terminal. On Mac: ⌘+Space → type "Terminal" → Enter. Run the blocks below one at a time.

1.1 — Install Claude Code

Mac:

```
brew install --cask claude-code
```

Anything else:

```
curl -fsSL https://claude.ai/install.sh | bash
```

Then sign in:

```
claude
```

1.2 — Install the Salesforce CLI

```
npm install -g @salesforce/cli
```

Or on Mac:

```
brew install salesforcedx/cli/sf
```

1.3 — Authenticate `sf` to your Salesforce org

Use a sandbox if you have one. Never point Claude at production on day one.

```
sf org login web --alias mysandbox --set-default
```

A browser pops up, you log in, done. Confirm it stuck:

```
sf org list
```

1.4 — Clone the marketing toolkit

```
git clone https://github.com/dustins89/cg-marketing-cli.git
cd cg-marketing-cli
python3 -m venv .venv
source .venv/bin/activate
pip install -e .
```

1.5 — Launch Claude in this directory

```
claude
```

That's it. Claude can now read and write files in your project, run `sf` commands against your authenticated org, and use the marketing CLIs to pull data from Google Ads, GA4, Search Console, Meta, and more.

Bonus — add the Salesforce MCP for cleaner SF tool access:

```
claude mcp add salesforce -- npx -y @salesforce/mcp \
  --orgs DEFAULT_TARGET_ORG --toolsets all
```

Restart Claude. Type `/mcp` to confirm it connected.

Step 2 — The first prompt to paste

This is the most important prompt in this packet. It sets six guardrail rules that **persist in Claude's memory** for every future session. Paste this whole block as your first message:

Save the following as feedback memories so you apply them in every future conversation in this project. Each should be its own memory file with a clear name and a "Why:" / "How to apply:" structure.

1. Always retrieve before deploy. Before any `sf project deploy` of a flow, permission set, profile, dashboard, custom object, or any metadata I already have locally, you must first run `sf project retrieve` for that component, diff the live version against my local copy, and show me the diff. Only deploy after I confirm.
Why: local files drift from the org within hours.
How: every deploy, every time, even if I sound in a hurry.
2. Ask explicit permission before any destructive or irreversible operation. Never run, without my typed confirmation:
 - sf project deploy (any component, any org)
 - sf project delete or --destructive-changes deploys
 - sf data delete or any DML deleting/updating records
 - sf apex run of anonymous Apex that writes data
 - Any deploy targeting a production org
 - git push, git reset --hard, git push --forceHow: state what you're about to do, name the org, wait for "yes".
3. Production deploys require a second confirmation. If the target alias contains "prod" or isn't clearly a sandbox, restate "Deploying to PRODUCTION org <alias>. Confirm again?" and wait for a second yes.
4. Never assume my local file matches the org. When I ask you to modify any metadata, your first step is to retrieve the live version and work from that. Do not edit local blind.
5. Ask who needs access after any access-controlled deploy. After deploying new Apex classes, fields, objects, permsets, or LWCs, ask which users/profiles need access before moving on.
6. "Don't change anything" means describe-only until I explicitly approve. If I tell you to investigate, audit, plan, or describe without making changes, stay read-only. Follow-up clarifying questions from me are NOT approval.

Save each as a separate file under feedback memory, link them from MEMORY.md, and confirm when done.

After you paste, Claude writes six files into `~/.claude/projects/<your-project>/memory/`. Every new session in that directory automatically loads them.

Test it. Open a new session and ask Claude to do a small deploy. If it doesn't pause and ask for confirmation, the rules aren't loading — check that `MEMORY.md` has the entries.

Step 3 — Build your own marketing CLI

If you want a marketing CLI for **your** stack — different platforms, different priorities — paste this into a fresh Claude Code session in an empty directory:

```
I want to build a personal marketing CLI in Python that I'll use with you to audit and manage my paid ads, SEO, and analytics. I'm modeling it after cg-marketing-cli. Architecture I want:
```

- One Python subpackage per platform (Google Ads, GA4, Search Console, Meta, Google Business Profile, CallRail, etc.)
- Each platform has `pull` commands (read-only, JSON or Rich tables) and `apply` commands (mutations, per-change y/N/q confirm, audit.log)
- All credentials live in a single `~/.marketing-cli/credentials.yaml` (chmod 600, gitignored)
- Python 3.9+, Rich for tables, Click or Typer for the CLI

```
Before we write any code:
```

1. Ask me which platforms I actually use today. We only build CLIs for those.
2. For each platform, ask whether I already have API access or need to set it up.
3. Walk me through getting API credentials for the highest-leverage platform, step by step.
4. Once I have credentials for at least one platform, scaffold the project structure: pyproject.toml, credentials file, shared core/ for auth + audit, and the first platform's subpackage.
5. Implement ONE `pull` command end-to-end so I see data flowing.

```
Do NOT make assumptions. Do NOT write code before step 4. Do NOT install anything without telling me what it does first. Treat this like a paid intake call.
```

```
Also: save the following as feedback memory so it persists into every future session in this project:
```

```
"For this project, never write code without my explicit go-ahead. Always describe what you're about to build, list the files that will be touched, and wait for me to type 'yes' before touching the filesystem."
```

You end up with a scaffolded project structurally compatible with this repo's patterns — same pull/apply split, same credentials layout, same audit log discipline. You can later copy specific platform packages from this repo if their interfaces line up.

Step 4 — What to learn before letting Claude touch a real org

Claude amplifies whatever Salesforce skill you have. It doesn't replace the fundamentals. Don't turn Claude loose on a production org until you have:

NON-NEGOTIABLE BASELINE

1. **Salesforce admin fluency.** Know what an object, field, record type, validation rule, flow, permission set, and profile actually are. Claude will produce metadata XML that looks plausible but is subtly wrong — if you can't read it, you'll deploy broken stuff. Trailhead Admin Beginner + Intermediate are the floor.
2. **Sandbox discipline.** Have a dev sandbox or scratch org. `sf config set target-org=mysandbox`.
3. **Git basics.** `status`, `diff`, `commit`, `revert`, `stash`. Claude edits files; git is how you roll back.
4. **SFDX source format.** `force-app/main/default/` mirrors the org's metadata tree.

STRONGLY RECOMMENDED

5. **Reading metadata XML.** You don't need to write it, just skim it and tell if it matches what you asked for.
6. **Apex + SOQL literacy** if touching code. At minimum: what a trigger is, what `[SELECT ... FROM Account]` does, what a governor limit is.
7. **The retrieve-before-deploy reflex.** The #1 source of broken deploys. Local files drift within hours.

THREE PROMPTS TO TRY FIRST

1. "Audit my Google Ads at the asset level. Cover impression share, ad strength per RSA, extension performance, hourly CPA. Use `gads pull` for every claim. Output as Tier 1 / Tier 2 / Tier 3 with one numbered fix per finding."

2. "Pull my top 20 GA4 events over the last 30 days. Tell me which should be marked as Key Events, and which look like garbage I should remove."

3. "Retrieve the Lead object from my Salesforce org. Walk me through how leads get from web form to qualified opportunity. Identify any flow steps that look fragile."

The "aha" moment is usually **#1** — watching Claude run twenty API calls and produce an audit you'd have paid an agency \$3K for.

github.com/dustins89/cg-marketing-cli

[Code](#) · [Sample audits](#) · [Updated PDF](#) · [Prompting guide](#)